



PRM393 · Mobile Programming · Lab 03

Firestore-Powered Journal Trend Analyzer

Mẫu báo cáo (draft) — các chỗ [...] và <!-- TODO --> cần bạn điền / chèn ảnh sau.
Xuất PDF: mở file này bằng Word / Google Docs / Typora / Pandoc rồi Export → PDF.

Field	Value
STUDENT	[Họ và tên]
STUDENT ID	[MSSV]
PLATFORM	Flutter · Android
APPLICATION	Journal Trend Analyzer 1.0.0
VALIDATION DATE	[Ngày nộp / ngày chạy test, vd: 17 July 2026]
DATA SOURCE	OpenAlex API
CLOUD PLATFORM	Firebase
GITHUB	[Link repo, vd: https://github.com/ ...]

1. Project Overview

The application helps users search a research topic and inspect publication, journal, keyword, and author trends. OpenAlex is the authoritative publication source; Firebase provides identity

and cloud capabilities. Four persistent navigation destinations—**Home**, **Journals**, **Keywords**, and **Profile**—match the assignment specification.

Metric	Value
Main tabs	4
Required Analytics events	7
Firebase services	6
E2E tests	11/11 (<i>điền kết quả thật sau khi chạy Patrol</i>)

Implemented scope

Area	Implemented capabilities	Status
Authentication	Google Sign-In, Firebase Authentication user profile, sign-out and auth gate	Complete / []
Home	Topic search, total publications, average citations, most active year, top journal/author/paper, trend chart and publication paging	Complete / []
Publication	Title, authors, year, journal, citations, DOI/original link and abstract	Complete / []
Journals	Source search, ranked results, contribution/citation statistics, trend and related publications	Complete / []
Keywords	Monthly/topic rankings, frequency bars, trend, related journals/publications and ranked authors	Complete / []
Profile	User identity, notifications, PDF export/history, Remote Config, Crashlytics and sign-out	Complete / []

Technology stack

Layer	Technology
Client	Flutter 3.41.9 · Dart 3.11.5 · Material 3
State & data	Riverpod (ViewModels) · Repository/Service · OpenAlex HTTP API (Dio)
Routing	go_router (StatefulShellRoute, 4 tabs)
Cloud	Firebase Auth, Storage, FCM, Analytics, Crashlytics, Remote Config
Quality	Patrol 4.x · SonarQube · flutter analyze
Reports	pdf package → Firebase Storage

2. Architecture and MVVM Implementation

The project uses an MVVM-compatible layered design. Flutter screens remain focused on rendering and user interaction. Riverpod notifiers/providers serve as ViewModels, repositories expose domain operations, and services isolate external systems.

View (Screens & widgets)

→ ViewModel (Providers / Notifiers)

→ Repository (Domain facade)

→ Services (OpenAlex / Firebase)

Layer	Main components	Responsibility
Model	Work, Author, JournalSummary, UserSettings, AppNotification, report models	Typed application data, OpenAlex parsing, bookmark/report snapshots
View	Login, Home, Journal, Keyword, Profile and detail screens	Responsive UI; loading, error, empty and data states; navigation
ViewModel	AuthViewModel, HomeViewModel,	Async orchestration, state

Layer	Main components	Responsibility
	JournalViewModel, ProfileViewModel, ...	transitions, validation and UI notifications
Data	PublicationRepository, OpenAlex (ApiClient), AnalyticsService, StorageService, ...	API communication, Firebase SDK access, PDF generation and error translation

Topic analysis flow

1. Normalize the query and record `search_topic` .
2. Load publications, journals and keywords asynchronously.
3. Load top authors, influential papers, yearly trend and citation average.
4. Publish progressive state updates to avoid blocking the dashboard.

Reliability decisions

- Bounded pages and request timeouts protect the mobile client.
- Future publication years are filtered using the device year.
- Errors are mapped to user-friendly, retryable states.
- PDF reports are scoped by Firebase UID in Storage (`report/{uid}/...`).

Project organization

```
lib/
├─ core/           # config, theme, router, network, shared widgets
├─ firebase/       # Auth, Analytics, Crashlytics, FCM, Remote Config, Storage
└─ features/
    ├─ auth/
    ├─ home/
    ├─ journal/
    ├─ keywords/
    ├─ profile/
    ├─ publication/
    └─ shared/
```

Each feature follows:

```
feature/  
├─ data/          # models, datasources, repository impl  
├─ domain/        # entities, repository interfaces, use cases  
└─ presentation/  # viewmodels, screens, widgets
```

3. User Interface and Research Analytics

The UI uses Material 3, shared typography/colors, responsive layouts and explicit loading/error/empty states. The dashboard aggregates OpenAlex data while drill-down screens preserve the exact publication or entity context.

Screenshots (*chèn ảnh thật*)

#	Screen	File / note
1	Login (Google Sign-In)	<!-- TODO: screenshots/01_login.png -->
2	Home dashboard (metrics + yearly trend)	<!-- TODO: screenshots/02_home.png -->
3	Publication detail	<!-- TODO: screenshots/03_publication.png -->
4	Journals list / journal detail	<!-- TODO: screenshots/04_journals.png -->
5	Keywords ranking / keyword detail	<!-- TODO: screenshots/05_keywords.png -->
6	Profile (identity, reports, Firebase demos)	<!-- TODO: screenshots/06_profile.png -->

Research metrics

Total publications, average citations, active year, top journal, top author and most influential paper are visible for the selected topic.

Drill-down navigation

Publication, journal, keyword and author selections open analytical detail screens with related entities and trends.

4. Firebase Integration and Analytics

Firestore service	Application use	Evidence
Authentication	Google account sign-in, user profile and sign-out	Profile screenshot · Patrol TC1 / TC11
Storage	Upload/list/open/delete PDF reports under <code>report/{uid}/...</code>	Storage console · Profile report count
Cloud Messaging	Foreground/background/opened handling, local display, FCM token, notification center	Profile Firestore verification panel
Analytics	Seven required events with contextual parameters	Firestore Analytics Events report
Crashlytics	Global fatal handlers, handled exception and deliberate test crash	Crashlytics console issues/trend
Remote Config	Dynamic <code>max_journals</code> / <code>max_keywords</code> (hoặc key tương đương trong app)	In-app values · Patrol Remote Config TC

Required Analytics event mapping

Event	Parameters	Trigger
<code>login</code>	<code>login_method = google</code>	Successful Google authentication
<code>search_topic</code>	<code>keyword</code>	Topic analysis starts

Event	Parameters	Trigger
view_publication	publication_title , publication_year	Publication detail opens
view_journal	journal_name	Journal detail opens
view_keyword	keyword	Keyword detail opens
export_pdf	topic	PDF generated and uploaded
logout	—	User signs out

Evidence: <!-- TODO: chèn ảnh Firebase Analytics DebugView / Events dashboard -->

5. Cloud Reports, Remote Config and Crash Monitoring

PDF report workflow

Dashboard snapshot → PDF bytes → Firebase Storage

The generated A4 report contains the selected topic, summary metrics, yearly publication trend, top journals and top publications. It is saved locally and uploaded with PDF metadata. The app exposes report history and downloadable URLs.

Evidence: <!-- TODO: chèn ảnh Firebase Storage bucket (thư mục report/) -->

Dynamic configuration and demos

Remote values, handled exception, test crash and FCM token action are available on the Profile screen.

Evidence:

<!-- TODO: chèn ảnh Profile – Remote Config values + Crashlytics buttons -->

Crashlytics evidence

Crashlytics production dashboard showing the deliberate test crash issue (e.g. `FirebaseCrashlyticsTestCrash`).

Safety note: “Handled exception” records a non-fatal diagnostic event and keeps the app running. “Test crash” intentionally terminates the app and is used only for Crashlytics verification.

Evidence: <!-- TODO: chèn ảnh Crashlytics Issues / Trends -->

6. Automated End-to-End Testing with Patrol

The final suite was executed on **[device/emulator, vd: Android 17 emulator-5554]** using the real app, Google account picker, OpenAlex API and Firebase project.

#	Scenario	Main verification	Result
1	Google Sign-In	Native account selection and Home navigation	Pass / []
2	Topic Search	Publication results displayed	Pass / []
3	Publication Details	Metadata, authors and abstract	Pass / []
4	Journals Navigation	Statistics and journal results	Pass / []
5	Journal Details	Total works, citations and analysis	Pass / []
6	Keywords Navigation	Frequency statistics and rankings	Pass / []
7	Keyword Details	Trend and ranked authors	Pass / []

#	Scenario	Main verification	Result
			]
8	Profile Navigation	Identity, reports, notifications and sign-out	Pass / []
9	PDF Export	Report generation and successful Storage upload	Pass / []
10	Remote Config	Both dynamic values displayed	Pass / []
11	Logout	Return to Google Sign-In screen	Pass / []

```
flutter analyze      → [kết quả, vd: No issues found]
patrol test -d <device> → [vd: 11 successful, 0 failed, 0 skipped]
```

Evidence: <!-- TODO: chèn ảnh / copy log kết quả Patrol Android test report -->

Test sources: `patrol_tests/` (`authentication_test.dart` , `publication_test.dart` , `journal_test.dart` , `keyword_test.dart` , `profile_test.dart` , `export_test.dart` , `remote_config_test.dart`).



7. AI-Assisted Review, Challenges and Lessons Learned

AI-assisted quality review (SonarQube / Cursor AI / ...) was used before submission.

Metric	Value
Quality Gate	[OK / Failed]
Open issues	[0 / n]

Metric	Value
Bugs	[0 / n]
Vulnerabilities	[0 / n]
Code smells	[0 / n]

Sample findings (*điền findings thật từ Sonar / AI review*)

Finding	Impact	Resolution	Status
[Finding 1 — vd: import order inconsistent]	[Impact]	[How you fixed it]	Fixed / Open
[Finding 2]	...	...	...
[Finding 3]	...	...	...

Evidence: <!-- TODO: chèn ảnh SonarQube Quality Gate / AI review summary -->

Challenges encountered

- OpenAlex rate limits and variable response time required bounded requests and retries.
- Native Google Sign-In and Android permission dialogs required Patrol native automation.
- Remote Config values can differ from local defaults, so tests must validate behavior rather than fixed data.
- Crashlytics fatal events require app restart before upload.
- *[Thêm challenge riêng của bạn nếu có]*

Lessons learned

- Layered ViewModel/MVVM code is easier to test and maintain.
- Cloud evidence must be verified in production reports, not only DebugView.
- E2E tests should map directly to assessment scenarios.
- Dynamic services require robust assertions and user-friendly fallback states.
- *[Thêm bài học riêng nếu có]*

Conclusion

The Journal Trend Analyzer satisfies the Lab 03 functional, Firebase, architecture and testing requirements. *[Điều chỉnh sau khi có kết quả Patrol + Sonar thật:]* All eleven required E2E workflows passed on Android, the final SonarQube Quality Gate passed, and the application demonstrates a maintainable production-style Flutter implementation.

Appendix — Checklist trước khi xuất PDF

- ☐ Điền họ tên, MSSV, ngày, link GitHub
 - ☐ Cập nhật cột Status / Result cho đúng thực tế
 - ☐ Chèn ≥ 6 screenshot UI
 - ☐ Chèn Analytics Events screenshot
 - ☐ Chèn Storage + Crashlytics + Remote Config screenshots
 - ☐ Chèn Patrol report / terminal log
 - ☐ Chèn SonarQube / AI review evidence (≥ 3 findings)
 - ☐ Rà lại tiếng Anh (hoặc dịch sang tiếng Việt nếu giảng viên yêu cầu)
 - ☐ Export PDF 5–10 trang (đề bài)
-